SoSConfigurator

Interface Design Description: generalManagement HTTP/TLS/JSON

Abstract
Source-based description of the SoSConfigurator support system and its registered services. Documentation edition 5.3.0 retains the implementation and wire versions defined by the current primary source; no runtime upgrade or deployment is implied.

Contents

1 Documentation edition and source scope **3**

2 Service definition and contract **3**

3 Operations and data models **3**

4 Security and failure behavior **3**

5 Security and access model **3**

6 Go and Rust implementation variants **4**

6.1 Go implementation 4

6.2 Rust implementation 4

7 Primary source references **4**

8 Revision history **4**

1 Documentation edition and source scope

This is documentation edition 5.3.0, dated 2026-11-09. No earlier `documentation/v5.2.2` set exists for this system. These documents were written from the primary Java source and secure configuration; no older system documentation was deleted or changed. Runtime/dependency/service versions remain source-defined: `SoSConfiguratorConstants.SYSTEM_VERSION` is 5.2.2 and service versions are 1.0.0. Documentation publication does not upgrade the runtime or deploy an SoS.

2 Service definition and contract

The system produces `generalManagement` version 1.0.0. The secure HTTPS base path is `/sosconfigurator/general/mgmt`. This interface is restricted by the common management filter; the secure standalone policy is Sysop-only. Browser console sessions are not a replacement for this management identity.

3 Operations and data models

get-log. POST `/sosconfigurator/general/mgmt/logs` consumes an optional JSON `LogRequestDTO` and returns `LogEntryListResponseDTO` with HTTP 200. LogRequestDTO has `pagination` (PageDTO), `from`, `to`, `severity` and `logger`. PageDTO fields are integer `page`, integer `size`, string `direction` and string `sortField`. Validation and defaults are those of the vendored common LogService/LogValidation.

The response has `entries` (array of LogEntryDTO) and integer `count`. An entry contains strings `logId`, `entryDate`, `logger`, `severity`, `message` and `exception`; the record omits empty values. Count and pagination must not be inferred from array length alone.

get-config. GET `/sosconfigurator/general/mgmt/get-config` accepts optional repeated `keys` query parameters and returns `KeyValuesDTO` with HTTP 200. KeyValuesDTO contains `map`, a string-to-string mapping. It exposes the common/system public configuration keys, not a secret-store export. The system-specific public keys include catalog file, status timeout, API whitelist and console port.

4 Security and failure behavior

Mutual TLS and the configured identity policy authenticate a secure caller. Common management authorization is applied independently of `sosConfiguration` grants. Invalid filters, missing identity, denied management rights and unavailable backing storage are governed by common validation/error handling; these two handlers do not introduce a bespoke SoS planning-error envelope. Response DTO null/empty omission follows the vendored record annotations.

5 Security and access model

The secure Arrowhead connector uses TLS with client certificates, outsourced identity verification and token-to-certificate binding. `SosAccessFilter` permits a `sosConfiguration` operation for a verified Sysop, a named `sos.api.whitelist` system or a ConsumerAuthorization operation grant. An authenticated identity alone is insufficient. General management remains governed by the common management filter.

The separate console authenticates a browser with a configured scrypt password hash and the `ah_sos_console` cookie. It applies Host/Origin checks and session/lockout rules. A console session does not become

an Arrowhead identity token. Static UI/session paths are console-only; the Arrowhead connector returns 404 for them. Console handling marks authenticated operator requests so the internal API can execute without pretending to be an mTLS system caller.

6 Go and Rust implementation variants

6.1 Go implementation

No `Eclipse-Arrowhead-go/ah5-SoSConfigurator-go` folder exists in the designated primary Go tree. Go local-cloud systems may be targets of a Java SoSConfigurator only if their catalog and inter-cloud implementation meet the policy and executor checks. This is not a Go implementation of SoSConfigurator.

6.2 Rust implementation

No `Eclipse-Arrowhead-rust/ah5-SoSConfigurator-rust` folder exists in the designated primary Rust tree. The policy rejects a catalogued implementation without the required Gatekeeper/GateTunnel inter-cloud capability; the mere presence of Rust local-cloud roles does not establish eligibility or implement this system.

7 Primary source references

All source paths below are relative to `Eclipse-Arrowhead-java/ah5-SoSConfigurator-java`.

- `src/main/java/eu/arrowhead/sosconfigurator/SoSConfiguratorSystemInfo.java` and `SoSConfiguratorConstants.java`: registration, names, versions and paths.
- `api/http/SosConfigurationAPI.java`, `api/http/GeneralManagementAPI.java`, `policy/SosRequest.java`, `catalog/Catalog.java`, `planner/Planner.java` and `service/SosConfigurationService.java`, under the same Java package: operations, DTOs, planning and persistence.
- `security/SosAccessFilter.java` and `console/ConsoleFilter.java` under that package: caller and connector boundaries.
- `config/application.properties`, packaged `src/main/resources/application.properties` and `SoSConfiguratorDefaults.java`: secure overrides versus development/source defaults.
- `vendor/common-utils` and `vendor/data-transfer-objects`: common management, security and native sysMonitor publisher contract.

8 Revision history

2026-11-09: Initial source-based LaTeX documentation edition 5.3.0. Review draft; document compilation and source traceability checked.